

Week-9

a) Demonstrate a simple example of Spring web MVC framework.

CODE:

Add these to pom.xml file:

```
<!-- JSP Support -->

<dependency><groupId>org.apache.tomcat.embed</groupId>
<artifactId>tomcat-embed-jasper</artifactId></dependency>

<dependency><groupId>jakarta.servlet</groupId>
<artifactId>jakarta.servlet-api</artifactId>
<scope>provided</scope></dependency>

<dependency><groupId>jakarta.servlet.jsp.jstl</groupId>
<artifactId>jakarta.servlet.jsp.jstl-api</artifactId></dependency>

<dependency><groupId>org.glassfish.web</groupId>
<artifactId>jakarta.servlet.jsp.jstl</artifactId></dependency>
```

Controller class:

```
package com.example.demo.controller;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;

@Controller

public class HomeController {

    @GetMapping("/")

    public String home() {

        return "home"; // JSP file name (home.jsp)

    }

}
```

Add these to application.properties:

```
spring.mvc.view.prefix=/WEB-INF/views/
```

```
spring.mvc.view.suffix=.jsp
```

home.jsp:

```
<%@ page contentType="text/html; charset=UTF-8" language="java" %>
```

```
<html>
```

```
<head>
```

```
<title>Welcome</title>
```

```
</head>
```

```
<body>
```

```
<h2>Hello, Spring Boot with JSP!</h2>
```

```
</body>
```

```
</html>
```

Output:



Hello, Spring Boot with JSP!

b) Illustrate how the database is connected in Spring Framework by using a simple CRUD application.

CODE:

pom.xml

```
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
<groupId>com.mysql</groupId>
<artifactId>mysql-connector-j</artifactId>
</dependency>
<dependency>
<groupId>javax.servlet.jsp</groupId>
<artifactId>javax.servlet.jsp-api</artifactId>
<version>2.3.3</version>
<scope>provided</scope>
</dependency>
```

Configure application.properties

JPA Configuration

spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect

MySQL DB connection

spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

spring.datasource.url=jdbc:mysql://localhost:3306/test?useSSL=false&serverTimezone=UTC

spring.datasource.username=root

spring.datasource.password=password

JPA settings

spring.jpa.hibernate.ddl-auto=update

spring.jpa.show-sql=true

spring.jpa.database-platform=org.hibernate.dialect.MySQL8Dialect

Create Entity Class

Student.java

```
package com.example.demo.model;
```

```
import jakarta.persistence.*;
```

```
@Entity
```

```
public class Student {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private Long id;
```

```
    private String name;
```

```
    private String email;
```

```
    public Long getId() {
```

```
        return id; }
```

```
    public void setId(Long id) {
```

```
        this.id = id; }
```

```
    public String getName() {
```

```
        return name; }
```

```
    public void setName(String name) {
```

```
        this.name = name; }
```

```
    public String getEmail() {
```

```
        return email; }
```

```
    public void setEmail(String email) {
```

```
        this.email = email;
```

```
    }}
```

Create Repository in repository package

StudentRepository.java

```
package com.example.demo.repository;

import com.example.demo.model.Student;

import org.springframework.data.jpa.repository.JpaRepository;

public interface StudentRepository extends JpaRepository<Student, Long> {

}
```

Create Controller

StudentController.java

```
package com.example.demo.controller;

import com.example.demo.model.Student;

import com.example.demo.repository.StudentRepository;

import org.springframework.beans.factory.annotation.Autowired;

import org.springframework.stereotype.Controller;

import org.springframework.ui.Model;

import org.springframework.web.bind.annotation.*;

@Controller

public class StudentController {

    @Autowired

    private StudentRepository repo;

    @GetMapping("/students")

    public String listStudents(Model model) {

        System.out.println(repo.findAll());

        model.addAttribute("students", repo.findAll());

        return "studentList" }

    @GetMapping("/students/new")

    public String showForm(Model model) {

        model.addAttribute("student", new Student());

        return "studentForm";

    }
```

```

@PostMapping("/students/save")
public String saveStudent(@ModelAttribute Student student) {
    repo.save(student);
    return "redirect:/students";
}

@GetMapping("/students/edit/{id}")
public String editStudent(@PathVariable Long id, Model model) {
    model.addAttribute("student", repo.findById(id).orElse(null));
    return "studentForm";
}

@GetMapping("/students/delete/{id}")
public String deleteStudent(@PathVariable Long id) {
    repo.deleteById(id);
    return "redirect:/students";
}
}

```

Create JSP Pages

studentList.jsp

```

<%@ page contentType="text/html; charset=UTF-8" %>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<html><head><title>Student List</title></head>
<body>
<h2>Students</h2>
<a href="/students/new">Add Student</a>
<table border="1">
<tr><th>ID</th><th>Name</th><th>Email</th><th>Actions</th></tr>
<c:forEach var="s" items="${students}">
<tr>
<td>${s.id}</td>
<td>${s.name}</td>

```

```

<td>${s.email}</td>

<td>

<a href="/students/edit/${s.id}">Edit</a>

<a href="/students/delete/${s.id}">Delete</a>

</td>

</tr>

</c:forEach>

</table></body></html>

studentForm.jsp

<%@ taglib uri="http://www.springframework.org/tags/form" prefix="form" %>

<%@ page contentType="text/html; charset=UTF-8" %>

<html>

<head><title>Student Form</title></head>

<body>

<h2>Student Form</h2>

<form:form method="POST" action="/students/save" modelAttribute="student">

<form:hidden path="id" />

<p>Name: <form:input path="name" /></p>

<p>Email: <form:input path="email" /></p>

<input type="submit" value="Save" />

</form:form>

</body>

</html>

```

Output:



Students

[Add Student](#)

ID	Name	Email	Actions
----	------	-------	---------



Student Form

Name:

Email:

Student Form

Student:

Email:

students

[Add Student](#)

ID	Student	Email	Actions
2	student	22	Edit Delete

Console Logs

```
Intercepted Request URI: /students
Hibernate: select s1_0.id,s1_0.email,s1_0.name from student s1_0
[com.example.demo.model.Student@2d2f6d78]
Hibernate: select s1_0.id,s1_0.email,s1_0.name from student s1_0
Intercepted Request URI: /students/edit/2
Hibernate: select s1_0.id,s1_0.email,s1_0.name from student s1_0 where s1_0.id=?
Intercepted Request URI: /students/save
Hibernate: select s1_0.id,s1_0.email,s1_0.name from student s1_0 where s1_0.id=?
Hibernate: update student set email=?,name=? where id=?
Intercepted Request URI: /students
Hibernate: select s1_0.id,s1_0.email,s1_0.name from student s1_0
[com.example.demo.model.Student@3149b612]
Hibernate: select s1_0.id,s1_0.email,s1_0.name from student s1_0
Intercepted Request URI: /students/delete/2
Hibernate: select s1_0.id,s1_0.email,s1_0.name from student s1_0 where s1_0.id=?
Hibernate: delete from student where id=?
Intercepted Request URI: /students
Hibernate: select s1_0.id,s1_0.email,s1_0.name from student s1_0
[]
Hibernate: select s1_0.id,s1_0.email,s1_0.name from student s1_0
```

WEEK-10

a) Create a simple example of a hibernate application using eclipse IDE.

CODE:

Update SpringHibernateDemoApplication.java with below code:

```
package com.example.demo;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import com.example.demo.entity.Student;
import com.example.demo.repository.StudentRepository;

@SpringBootApplication

public class SpringHibernateDemoApplication implements CommandLineRunner {

    @Autowired

    private StudentRepository studentRepository; // Spring injects this for you

    public static void main(String[] args) {
        SpringApplication.run(SpringHibernateDemoApplication.class, args);
    }

    @Override

    public void run(String... args) throws Exception {

        // TODO Auto-generated method stub

        Student student = new Student("Alice");

        studentRepository.save(student);

        System.out.println("Students saved: " + student.getName());

    }

}
```

Create Student.java Entity class in com.example.demo.entity package:

```

package com.example.demo.entity;

import jakarta.persistence.*;

@Entity
@Table(name = "students")

public class Student {

    @Id

    @GeneratedValue(strategy = GenerationType.IDENTITY)

    private Long id;

    private String name;

    public Student() {}

    public Student(String name) {

        this.name = name;

    }

    public Long getId() { return id; }

    public String getName() { return name; }

    public void setName(String name) { this.name = name; }

}

```

Create StudentRepository.java class in com.example.demo.repository package:

```

package com.example.demo.repository;

import org.springframework.data.jpa.repository.JpaRepository;

import com.example.demo.entity.Student;

public interface StudentRepository extends JpaRepository<Student, Long> {

    // Optional: custom query methods

}

```

Update the application.properties file with below code:

```

spring.application.name=spring-hibernate-demo

# Database connection

spring.datasource.url=jdbc:mysql://localhost:3306/hibernatedb

```

spring.datasource.username=root

spring.datasource.password=password

spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

Hibernate properties

spring.jpa.hibernate.ddl-auto=update

spring.jpa.show-sql=true

spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect

Update the pom.xml file and add below dependencies to the dependencies section:

```
<dependency>
```

```
<groupId>org.springframework.boot</groupId>
```

```
<artifactId>spring-boot-starter-data-jpa</artifactId>
```

```
</dependency>
```

```
<dependency>
```

```
<groupId>org.springframework.boot</groupId>
```

```
<artifactId>spring-boot-starter-web</artifactId>
```

```
</dependency>
```

```
<dependency>
```

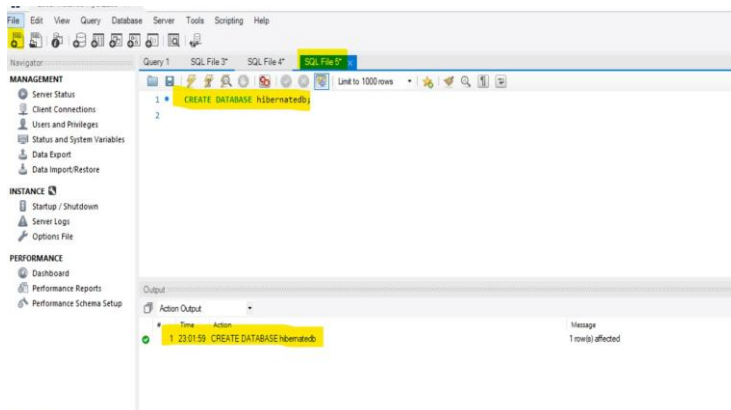
```
<groupId>com.mysql</groupId>
```

```
<artifactId>mysql-connector-j</artifactId>
```

```
<scope>runtime</scope>
```

```
</dependency>
```

Create new database in MySQL Workbench:



Output:

The screenshot displays two windows from an IDE. The top window shows the console output of a Spring Boot application. The bottom window shows a database client interface with SQL queries and their execution results.

Console Output:

```
Database driver: undefined/unknown
Database version: 8.0
Autocommit mode: undefined/unknown
Isolation level: undefined/unknown
Minimum pool size: undefined/unknown
Maximum pool size: undefined/unknown
2025-10-21T23:03:36.987+05:30 INFO 42548 --- [spring-hibernate-demo] [main] o.h.e.t.j.p.i.JtaP
Hibernate: create table students (id bigint not null auto_increment, name varchar(255), primary key (id))
2025-10-21T23:03:37.084+05:30 INFO 42548 --- [spring-hibernate-demo] [main] j.LocalContainerEn
2025-10-21T23:03:37.264+05:30 WARN 42548 --- [spring-hibernate-demo] [main] JpaBaseConfigurati
2025-10-21T23:03:37.570+05:30 INFO 42548 --- [spring-hibernate-demo] [main] o.s.b.w.embedded.t
2025-10-21T23:03:37.577+05:30 INFO 42548 --- [spring-hibernate-demo] [main] c.e.demo.SpringHib
Hibernate: insert into students (name) values (?)
Student saved: Alice
```

Database Client Window:

Query 1 SQL File 3* SQL File 4* SQL File 5* x

```
1 CREATE DATABASE hibernatedb;
2 use hibernatedb;
3
4 select * from students;
```

Result Grid:

id	name
1	Alice

students 1 x Apply

Output:

Action Output

#	Time	Action	Message
1	23:01:59	CREATE DATABASE hibernatedb	1 row(s) affected
2	23:04:22	use hibernatedb	0 row(s) affected
3	23:04:33	select * from student LIMIT 0, 1000	Error Code: 1146. Table 'hibernatedb.student' doesn't exist
4	23:04:39	select * from students LIMIT 0, 1000	1 row(s) returned

No object selected

b) Create an application to demonstrate Hibernate Query Language.

CODE:

Update SpringHqlDemoApplication.java with below code:

```
package com.example.demo;

import com.example.demo.entity.Student;
import com.example.demo.repository.StudentRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import java.util.List;

@SpringBootApplication

public class SpringHqlDemoApplication implements CommandLineRunner {

    @Autowired
    private StudentRepository studentRepo;

    public static void main(String[] args) {
        SpringApplication.run(SpringHqlDemoApplication.class, args);
    }

    @Override
    public void run(String... args) {
        // Insert test data
        studentRepo.save(new Student("Alice"));
        studentRepo.save(new Student("Bob"));
        studentRepo.save(new Student("Alice"));

        // HQL: Find students by name
        System.out.println(" Students named 'Alice':");
        List<Student> alices = studentRepo.findByName("Alice");
        for (Student s : alices) {
```

```

System.out.println("ID: " + s.getId() + ", Name: " + s.getName());
}

// HQL: Count students
long count = studentRepo.countStudents();

System.out.println(" Total Students: " + count);
}
}

```

Create Student.java Entity class in com.example.demo.entity package:

```

package com.example.demo.entity;

import jakarta.persistence.*;

@Entity
@Table(name = "students")

public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    public Student() {}

    public Student(String name) {
        this.name = name;
    }

    // Getters & Setters

    public Long getId() { return id; }

    public String getName() { return name; }

    public void setName(String name) { this.name = name; }

}

```

Create StudentRepository.java class in com.example.demo.repository package:

```

package com.example.demo.repository;

```

```

import com.example.demo.entity.Student;

import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.CrudRepository;
import java.util.List;

public interface StudentRepository extends CrudRepository<Student, Long> {

    // HQL: Get students by name
    @Query("FROM Student WHERE name = :name")
    List<Student> findByName(String name);

    // HQL: Count total students
    @Query("SELECT COUNT(s) FROM Students")
    long countStudents();

    // HQL: Custom delete by name (optional)
    // @Modifying
    // @Transactional
    // @Query("DELETE FROM Student s WHERE s.name = :name")
    // void deleteByName(String name);
}

```

Update the application.properties file with below code:

```

spring.application.name=spring-hibernate-demo

# Database connection
spring.datasource.url=jdbc:mysql://localhost:3306/hibernatedb
spring.datasource.username=root
spring.datasource.password=password

# Hibernate properties
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect

```

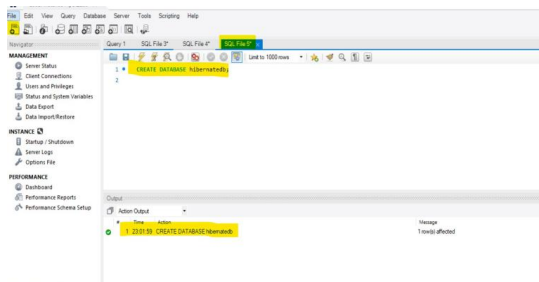
Update the pom.xml file and add below dependencies to the dependencies section:

```

<dependency><groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-data-jpa</artifactId></dependency>
<dependency><groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId></dependency>
<dependency><groupId>com.mysql</groupId>
<artifactId>mysql-connector-j</artifactId>
<scope>runtime</scope></dependency>
<dependency>
<groupId>jakarta.persistence</groupId>
<artifactId>jakarta.persistence-api</artifactId>
</dependency>

```

Create new database in MySQL Workbench:



Output:

